POO (Java) : TP1

Exercice 1 : Gestion d'une bibliothèque

Créez une classe **Livre** avec les attributs suivants :

- titre (String)
- auteur (String)
- isbn (String)
- disponible (boolean, initialisé à true)

La classe doit contenir :

- Un constructeur
- Des getters/setters
- Une méthode emprunter() qui met l'attribut « disponible » à false (retourne true si succès, false si déjà emprunté)
- Une méthode retourner() qui remet l'attribut « disponible » à true
- Une méthode toString()

Créez ensuite une classe **Bibliotheque** qui contient un tableau simple de livres (avec une taille maximale fixe), avec :

- Une méthode ajouterLivre(Livre livre) qui gère le cas où le tableau est plein
- Une méthode emprunterLivre(String isbn) qui cherche le livre par ISBN et l'emprunte
- Une méthode afficherLivresDisponibles().

Enfin, dans une classe **Main**, testez le tout.

Exercice 2 : Compte bancaire simple

Créez une classe **CompteBancaire** avec :

- Attributs privés : numeroCompte (String), titulaire (String), solde (double)
- Un constructeur qui initialise le solde à 0
- Des méthodes deposer(double montant) et retirer(double montant) avec validation :
    - Le montant doit être strictement positif
    - Le retrait ne peut pas dépasser le solde
- Une méthode afficherSolde()
- Des getters pour numeroCompte, titulaire et solde

Créez une classe **Banque** qui gère un tableau simple de comptes (taille maximale fixe) avec :

- Une méthode ouvrirCompte(CompteBancaire compte)
- Une méthode rechercherCompte(String numeroCompte) qui retourne le compte ou null
- Une méthode effectuerVirement(String numSource, String numDest, double montant) qui transfère de l'argent entre deux comptes
- Une méthode afficherTousLesComptes()

Testez avec une classe **Main**.

Exercice 3 : Application console

Écrivez un programme Java « compact » qui :

- Ne déclare pas de classe englobante explicite (fichier source compact).
- Utilise void main() comme méthode d'entrée d'instance (pas de static, pas de String[] args).
- Utilise la nouvelle classe IO du package java.lang pour lire une ligne saisie par l'utilisateur et afficher un message personnalisé.
- Calcule et affiche la somme des entiers de 1 à N, où N est fourni par l'utilisateur.

Remarque : En Java 25, les fichiers source compacts importent automatiquement java.base et la classe IO est désormais dans java.lang .

Exercice 4 : Validation dans un constructeur avec Flexible Constructor Bodies

Créez une classe Produit qui :

- Possède un champ prix (double) et un champ nom (String).
- Le constructeur doit valider que le prix est strictement positif avant d'initialiser les champs et avant d'appeler un autre constructeur ou super().
- Utilisez la fonctionnalité Flexible Constructor Bodies pour effectuer la validation dans le « prologue » du constructeur.
- Ajoutez un second constructeur qui délègue au premier via this(...).

Exercice 5 : Module Import Declarations pour simplifier les dépendances

Créez deux fichiers source :

- Geometrie.java : un fichier compact qui utilise import module java.base; (nouveauté Java 25) et définit une méthode void main().
- Dans ce programme, calculez le périmètre et l'aire d'un cercle de rayon donné, en utilisant Math.PI et en formatant le résultat avec String.format.

Puis, dans un second fichier compact Statistiques.java, importez le module java.base et calculez la moyenne d'un tableau d'entiers prédéfini.

Exercice 6 : Calcul de statistiques sur les températures

Écrivez un programme Java qui :

- Demande à l'utilisateur combien de températures il souhaite saisir (entre 3 et 30).
- Saisit ensuite chaque température (valeurs réelles entre -50 et 60 °C). Si une valeur est hors plage, redemander.
- Calcule et affiche :
    - La **moyenne** des températures
    - La **température maximale** et la **minimale**
    - L'**amplitude thermique** (max - min)
    - L'**écart-type** (formule : $\sqrt{(\Sigma(x_i - \text{moyenne})^2 / n)}$)
    - Le **nombre de jours de gel** (température < 0)
    - La **température médiane** (sans utiliser de collection ni tri automatique — utilisez un tableau et un tri manuel type tri à bulles)

Exemple d'exécution attendue :

Combien de températures ? 5

Température 1 : 12.5

Température 2 : -3.2

Température 3 : 8.0

Température 4 : 15.7

Température 5 : 0.0

--- Résultats ---

Moyenne : 6.60 °C

Maximum : 15.70 °C

Minimum : -3.20 °C

Amplitude : 18.90 °C

Écart-type : 6.83

Médiane : 8.00 °C

Jours de gel : 1

Exercice 7 : Etudiants admis

Vous devez développer un petit programme de gestion de notes d'étudiants.

1. **Créer un Record Student** avec deux composants :
   - String name
   - double average (la moyenne de l'étudiant)
2. **Créer la classe principale** avec une méthode main qui initialise une liste de plusieurs étudiants (avec des moyennes inférieures, égales et supérieures à 10).
3. **Implémenter le traitement :**
   - Méthode 1 : Calculer la moyenne et afficher les étudiants ayant la moyenne sans l'utilisation de l'API Stream.
   - Méthode 2 : Utilisez l'API Stream pour filtrer uniquement les étudiants ayant la moyenne (c'est-à-dire average >= 10.0).
   - Nouveauté Java 25 : Utilisez le nouveau collecteur Gatherers.windowFixed(...) ou un Gatherer personnalisé / intermédiaire pour regrouper ou traiter les étudiants. *Pour simplifier et mettre en valeur les nouveautés récentes (Java 21–25), vous utiliserez le Pattern Matching pour les Records combiné à la syntaxe modernisée des Flexible Main Methods qui permet d'alléger la méthode main sans déclaration explicite de classe si souhaité, ou d'utiliser les Gatherers.*

Objectif : Filtrer la liste, extraire uniquement les étudiants admis et afficher un message clair du type : "L'étudiant [Nom] est admis avec une moyenne de [Moyenne]/20."